

GET LUCKY GOLF

Insurance-backed hole-in-one gaming

Creative Direction & App Redesign Brief

A full redesign of the Get Lucky Golf player app for public beta. The app that exists today is a reference for what the product does — not for how it should look, and not for how it should be put together.

Prepared by Johannes Le Roux, Founder

Contact johannes@getluckygolfclub.com

Date 25 August 2026

Status Working beta — the app functions end to end; the visual layer has never been designed

Engagement Full redesign. Get Lucky V2.

1. Project snapshot

Get Lucky Golf is a mobile-first web app that lets golfers place an insurance-backed bet on any par-3: stake R50–R1,000 on a hole-in-one and win up to R1,000,000 if they sink an ace. Indwe Risk Services (FSP 3425) is the headline sponsor and insurer, with Santam as co-insurer. The product is South-Africa-first: ZAR, en-ZA, PayFast payments, 18+, SA residents only.

The player journey is: pick a course and a par-3 → choose a stake tier → pay → film the tee shot in the app → declare the result → if it went in, upload a video, a course certificate and a 4-ball affidavit → manual verification → payout.

Where we are. A genuinely working beta. The full player journey, an admin back-office, membership status and the payments and verification plumbing are built and deployed on Vercel. A closed beta round has already run. Separately, an engineering brief covers the hardening needed to run real money at public scale.

What this brief covers is different. The app works. It does not look like it is worth R1,000,000 of anyone's trust. The visual layer grew out of the build rather than out of a design, and it now needs a creative director.

1.1 What we are asking for

A full redesign of the player-facing app. A visual language, not a coat of paint: layout, structure, hierarchy, components, iconography, motion, and how the brand expresses itself in a product rather than on a poster.

27 surfaces, which are a **functional checklist** rather than a set of layouts to preserve — you may propose merging, splitting or adding screens. Flat comps plus two short spreadsheets and a one-page note per screen. A logo set including an icon-only mark that does not exist today. An icon set to replace the emoji currently doing that job.

The brand is open too. See section 9.

1.2 Tech context (for orientation only — no code required from you)

Layer	Technology
Framework	Next.js 16 (App Router), React 19, TypeScript
Styling	One global stylesheet plus design tokens; Tailwind v4 installed
Fonts	Poster Gothic Round ATF (display), Inter (body), Space Mono (numerals)
Data / auth	Supabase, Google sign-in
Payments	PayFast — hosted checkout, off-site
Hosting	Vercel

You will not touch code. The design pack is structured so that your files can be handed straight to the engineering side and implemented from.

2. What exists today — an honest assessment

Screenshots of every reachable screen are in [design/00-reference/current-app/](#). Look at those before reading further — **as a specification of what the product does, not as a design.** They show you every function that has to survive. They are not a layout to improve on, and you should feel no obligation to any composition in them.

The rest of this section is what a technical audit of the front end found. It is the reason this engagement exists, and it is diagnosis rather than inheritance.

2.1 The base is the Get Lucky website, not this app

[getluckyjo/getlucky-www](#) is the design base for V2. Its [src/app/globals.css](#) is the cleanest expression of the brand we have: eleven colour tokens and two font families, with the Tailwind theme deriving from a single `:root` block instead of duplicating it. Fifty-nine lines, and it is right.

```
--background #f5f0e1  --green      #335231  --gold      #c9a94e
--foreground #1a1a1a  --green-light #4a7a3d  --gold-light #e8d48b
--cream      #f5f0e1  --green-dark #1e3120  --charcoal  #2a2a2a
--cream-dark #e8e0cc  --charcoal-light #3a3a3a
```

Display is Poster Gothic Round ATF at weight 800, self-hosted; body is Inter at 300–700. Focus rings are 2px gold at 2px offset. A copy of the file is in [design/01-tokens/getlucky-www-globals.css](#), and the token sheet is built from it.

The mood is "accessible luxury": prize numbers set large, gold on green, course photography, paired with insurance-backed trust signals.

The palette is open. This is Get Lucky V2, not a tidy-up — you are free to repaint any colour. The website values are the starting point, not a constraint. What the app should inherit regardless is the *architecture*: one source of truth, tokens that are actually used.

2.2 But the design system is not actually enforced

The app has drifted from the website and then drifted from itself. It declares a clean set of design tokens and largely ignores them. **55 distinct hardcoded colour values appear across 45 files — roughly 350 occurrences.** The brand green is written out as a literal 49 times rather than referenced as a token. A legacy red (`#c0392b`) appears 23 times and contradicts the official red. There are off-palette greens and golds that exist for no reason anyone can now recall.

It has also drifted from the website's naming: the app calls the brand green `--green-deep` where the website calls it `--green`, and it references `--font-heading`, a variable the website defines and the app never did — so it currently resolves to nothing.

The practical consequence: **changing a colour today changes almost nothing.** This is being fixed on the engineering side while you design — see section 8 — so that your palette lands in one place rather than 45, under the website's naming.

2.3 Thirteen emoji are doing the work of an icon set

The Play button at the centre of the main navigation is 🏏. The magnifying glass in the search field is 🔍 injected through CSS. There are eleven more. These render as a different picture on iPhone, Android, Windows and Mac, cannot be recoloured or aligned precisely, and read as unfinished to anyone who notices.

2.4 The app is not using the real logo

The actual Get Lucky mark — hand-lettered cream script with a forest-green shadow, over "GOLF" in a condensed sans — lives in the website repository, along with a properly drawn reversed variant for dark backgrounds and a separate challenge lockup. It is good work.

The app is not using it. `public/logo.svg` in the app is an unrelated machine-traced path filled emerald `#007728`, and what actually ships is a 292 KB PNG. That file should be deleted rather than redrawn.

Two real gaps remain even with the website's assets:

- **No vector master anywhere.** Every logo file in either repository is a PNG at roughly 2932 x 2195. There is no `.ai`, `.svg` or `.eps`.
- **No icon-only mark.** The lockup is script lettering — excellent at 200px, unreadable at 24px. A mobile app needs a mark for the tab bar, the favicon, the app icon and the avatar. One does not exist, and drawing it is the highest-value single thing in this engagement.

2.5 There are real accessibility failures

Measured, not estimated:

Combination	Contrast	Verdict
Gold <code>#c9a94e</code> on cream	2.12:1	Fails badly. Gold is used as text on cream today.
Gold on green <code>#335231</code>	3.87:1	Fails for body text. Visible in the stake multiplier chips.
Gray <code>#999</code> on cream	2.66:1	Fails — and it is the secondary text colour in 12+ places.
Green <code>#335231</code> on cream	8.20:1	Good.

Reduced-motion preferences are honoured for a deleted marketing page but not for any of the fifteen animations in the app itself.

2.6 The desktop experience is a phone in a bezel

Every screen renders inside a literal simulated iPhone — black chassis, notch, camera dot — floating on a near-black page. There is no desktop layout behind it. This is the single loudest "this is a prototype" signal in the product.

2.7 What the build can already do

None of this is binding. It is what the engineering side already supports, so you know what is cheap rather than what is expected.

- **Continuous fluid scaling.** Type, spacing and radius scale smoothly from a 320px phone to a tablet with no breakpoint jumps. If you want that, it already works. If your design wants fixed steps instead, say so.

- **A motion system.** A consistent expo-out curve with staggered entrances is in place. Restrained, and easy to extend or replace.
- **Large display type in gold on green.** When the app shows a prize number it briefly looks like the product it wants to be. That is the one moment worth studying — not to copy, but because it is evidence of what the brand can do when it is given room.

3. Scope

3.1 In scope — a full redesign

- **The player app: 27 surfaces**, listed in section 5 and in [design/00-reference/screen-inventory.md](#). They are a **functional checklist**, not a set of layouts.
- **The whole visual language.** Colour, type, spacing, components, iconography, imagery, motion, and the structure and hierarchy of every screen.
- **How the screens are composed.** You may propose merging two screens into one, splitting one into two, or adding a step. See 3.4.
- **The brand's digital expression**, including the wordmark itself. See section 9.
- **A logo set** — vector masters, and the icon-only mark that does not exist today.
- **An icon set** to replace the thirteen emoji currently doing interface work.
- **A photographic treatment recipe** for course imagery.
- **One desktop backdrop comp.**

3.2 Out of scope

- **The admin back-office** — nine screens used by staff for claims adjudication. Deliberately left as-is for beta.
- **The PayFast checkout page.** Payment happens off-site on PayFast's own hosted page. We cannot design it.
- **The marketing website** at getluckygolf.co.za. It will follow the app, not lead it — see section 9.
- **New photography.** Use the 19 course photographs already in the repo.
- **A responsive desktop application layout.** One backdrop comp only.
- **Code.** You design; the engineering side builds.

3.3 The functional contract

This is what has to remain true. Every item states a **capability that must exist and stay reachable** — none of them says where it sits, what it looks like, or how many screens it takes. That part is yours.

#	Must remain true	Why
1	A user cannot reach real-money play without passing a hard 18+ gate that captures date of birth and an explicit consent. It cannot become a dismissible banner.	Legal requirement.
2	The Indwe sponsor presence is visible throughout the app.	Contractual. Its size, placement and treatment are open — see 4.6.
3	Terms, Privacy and Responsible Play stay reachable, and the National Responsible Gambling link stays on the under-18 refusal.	Regulatory posture.
4	The six stake tiers keep their exact amounts. No rounding, no invented tiers.	Server-validated; they are the pricing.
5	Every function in section 5 survives and stays reachable.	It is the product.
6	Payment hands off to PayFast and comes back.	The handoff is fixed; the screens either side are yours.
7	Minimum tap target 44 x 44pt.	Usability floor.
8	Text contrast at least 4.5:1, or 3:1 above 24px.	See 2.5.

The six tiers, for reference: R50 → R25,000 · R100 → R60,000 · R150 → R100,000 · R250 → R200,000 · R500 → R500,000 · R1,000 → R1,000,000.

3.4 Restructuring the flow — propose it, don't assume it

The 27 screens exist because today's app has 27 screens, not because that is the right number. If the play loop should be four steps instead of six, or the two onboarding screens should be one, say so.

But propose it before you draw it. Write it up in `design/00-reference/flow-proposal.md` — what merges into what, what is new, what goes away, and why. It is a **Stage 0 deliverable**, alongside the concept directions.

The reason is cost, not caution. Changing what a screen looks like is a day of engineering. Changing how many steps sit between "choose a stake" and "money has moved" touches payment state, verification and the bet record, and it needs to be estimated rather than discovered. A proposal gets a yes, a no, or a counter — quickly. A surprise in wave three does not.

4. How the handback works

This section is the operational core of the brief. Read it before you start drawing.

Everything goes in the `design/` folder, which is already built and pre-filled. Open `design/README.md` first — it is the one-page version of this section.

4.1 The principle: blank means unchanged

Every sheet and every screen card arrives already populated with what the app does today. You edit only what you are changing. A token sheet with eight edited cells is a complete and correct deliverable. Nothing here asks you to fill in a blank form.

4.2 The canvas

375pt wide, always. Height is whatever the content needs, rounded to a multiple of 8 — most screens scroll, so do not force everything into one viewport height.

The app is built on a fluid scale, so what you design at 375 stretches smoothly to about 480 with no second design needed. **You never design a tablet or a desktop layout.** Safe areas, reserved regions and font constraints are in `design/00-reference/device-spec.md`.

4.3 Comps

PNG, 750px wide (2x the canvas), sRGB, flattened, under 1.5 MB each. 750px is comfortably enough to read type sizes and measure spacing off the image.

Named `NN-slug__state.png`, dropped into the matching folder in `design/02-screens/`:

```
design/02-screens/16-home/16-home__default.png
design/02-screens/16-home/16-home__empty.png
design/02-screens/08-choose-stake/08-choose-stake__error.png
```

`state` comes from a closed list of nine words, given in section 5.2. A closed list is what keeps a folder from filling up with `__hover-2-final-FINAL.png`.

For each screen and state, exactly one file has no `--` suffix, and that file is the one we build. Anything suffixed is commentary. Exploration you want seen but not built gets `--alt-a`, `--alt-b`.

4.4 The redline twin

Export a second copy of each default comp with **numbered magenta circles** on it, saved as `16-home__default--redline.png`. The numbers key to the notes in your screen card.

In a reskin this would be optional. In a redesign it is the main thing: almost nothing maps onto the old screen, so the numbered notes are how you explain the new structure at all. Required on everything except `ABSORBED` and `CARRIED-OVER`.

That is all the annotation we need. No measurements, no specs, no callout syntax to learn — a layer of numbered circles, so the image and the words can point at each other.

4.5 The screen card

One markdown file per screen, already sitting in each folder, pre-filled with what that screen does today. `design/02-screens/_EXAMPLE.md` is a completed one — **read it before filling in your first.**

Four of its sections carry real weight, and each exists because a flat image cannot carry that information:

- **Change type** — one word that tells the engineering side what kind of job this screen is. The five values are in the table below.

- **Added / Removed / Kept** — because **removal is invisible in an image**. If you delete a paragraph, the comp simply does not show it, and that is indistinguishable from you forgetting it.
- **Final copy** — because your comp will contain realistic placeholder text. Anything not listed here is treated as placeholder and left wired to live data. Without this we would commit "Leopard Creek Country Club" as literal text and break the database binding behind it.
- **Fixed vs scrolls, and which blocks repeat** — neither is visible in a still image, and getting either wrong puts content behind the tab bar or replaces a live list with six hardcoded rows.

The change-type values:

Value	Means
REDESIGNED	New design, same functional scope, still one screen. The usual answer.
RECOMPOSED	Absorbs, splits or reorganises other screens. Add a Replaces: line naming their numbers, and reference the flow proposal.
NEW	A screen that does not exist today. Put it in a folder named new-<slug> .
ABSORBED	This screen goes away and its function moves elsewhere. Add an Absorbed into: line, and deliver no comps for it.
CARRIED-OVER	You are deliberately keeping today's structure. Rare — say why.

4.6 The two spreadsheets

design/01-tokens/tokens.csv — colour, spacing, radius, shadow, motion. About 38 rows, pre-filled. **design/01-tokens/type-scale.csv** — nine type roles.

CSV rather than a design tool export, so they open in Numbers or Excel and you can see them as tables. Give sizes in pixels at 375 width; the responsive scale is derived from that, so you never write a breakpoint or a **clamp()**.

Each row is marked with its **source**: **getlucky-www** for values taken from the website, **app-only** for things the app needs and the website has no equivalent for — error states, the bottom tab bar, motion, the fluid spacing scale.

design/01-tokens/README.md explains both sheets, and carries the contrast table you should read before committing to gold anywhere.

One thing worth your attention while you are in there: the bottom tab bar currently takes **120pt of an 812pt screen — nearly 15% of the viewport** — because the sponsor banner sits stacked above five tabs. The banner must stay visible; its height is yours to solve. Every screen in the app gains room if you can.

4.7 Assets

design/03-assets/ASSETS.md has the full specification: the logo set, the 19-icon set with exact export settings, and the imagery treatment recipe. These two sets are the highest-value items in the engagement, because both are genuinely broken rather than merely dated.

4.8 Work in progress

`design/04-inbox/` is ignored by version control. Nothing there will ever be built by accident. Move a file out of the inbox when it is ready to be looked at.

5. Screen inventory

This is a functional checklist, not a layout. Each row is a job the product has to do and a set of states it has to handle. Whether that job needs its own screen, shares one, or splits into two is a design decision — see 3.4, and put it in the flow proposal.

The full table, with the source file and current purpose of each, is in `design/00-reference/screen-inventory.md`. Summary by group:

Group	Screens	Count
Onboarding and auth	Splash, onboarding carousel, age check, auth, payment setup	5
The play loop	Select course, challenge-hole panel, choose stake, stake confirmation, payment return, record, confirm, claim, miss, verify	10
Main app	Home, leaderboard, history, account, membership	5
Chrome and system	Tab bar, legal template, 404, error, loading, toast	6
Desktop	Desktop backdrop	1

49 comps against today's screen map, plus a redline twin for each. That number moves if your flow proposal changes the composition — which is fine, and is exactly why the proposal comes first.

5.1 The five that matter most

If attention has to be unevenly distributed, distribute it here. Each is named by what it does, not by the screen it currently occupies:

1. **Committing money.** Six tiers, the prize number as hero. Where the product either feels credible or does not.
2. **Coming back.** What a returning player sees, and what it invites them to do next.
3. **The ace.** The single best moment the product can deliver, currently rendered as a trophy emoji.
4. **The miss.** Far more common, and where the tone decides whether anyone plays twice.
5. **Understanding the offer.** Explaining an unfamiliar and slightly improbable proposition to someone who has never heard of it.

5.2 The state vocabulary

`default`, `empty`, `loading`, `error`, `success`, `selected`, `modal`, `permission`, `blocked`. The inventory lists which states each function has to handle. If your composition changes where a state lives, move it and note it — the requirement is that every state is designed somewhere, not that it stays where it is today.

Empty and error states matter more here than in most products. A new user meets the empty state first, and an error on a screen where money has just moved is the moment trust is won or lost.

6. Deliverables and acceptance criteria

6.1 Package acceptance

A package is accepted when it passes `npm run check:design`, an automated check that validates:

1. Every screen folder matches the inventory.
2. Every required state has a comp, with no state names outside the closed vocabulary.
3. Exactly one unsuffixed file per screen and state.
4. Every comp is PNG, 750px wide, under 1.5 MB.
5. Every screen folder has a filled-in card with all required sections and a valid change type.
6. Every screen has a redline twin, except `ABSORBED` and `CARRIED-OVER`.
7. Every `RECOMPOSED` screen names what it `Replaces:`, and every `ABSORBED` screen names what it is `Absorbed into:`.
8. Both token sheets parse, with no renamed, reordered or deleted rows, and valid values.
9. Every icon SVG is 24x24 with no embedded raster or gradients.

A checklist a person runs is a checklist a person skips. This one is a script, and it runs before anyone reviews anything — so a package either passes or comes back with a specific list.

6.2 Implementation acceptance

Per screen, on the engineering side:

- Renders correctly at 375, 430 and 768px.
- Reviewed side by side against the comp and signed off by you.
- Every declared state built and reachable.
- Zero hardcoded colours; everything through tokens.
- Zero emoji in the player app; every glyph from the icon set.
- Every function from section 5 present and reachable; any composition change matches the agreed flow proposal.
- No live data replaced by placeholder text from a comp.
- Contrast at least 4.5:1; tap targets at least 44pt.
- Reduced-motion preferences honoured.
- Build, lint and tests pass.

7. Phasing — directions first, then depth, then breadth

7.1 Stage 0 — concept directions

Before anything is resolved, we want two or three whole-app visual directions.

Each direction is the *same two screens* — committing money, and coming back — drawn in a genuinely different visual language. Not three colourways of one idea: three answers to "what should this product feel like". Alongside them:

- A short written rationale per direction. What it is going for, who it is aimed at, what it sacrifices.
- **The flow proposal** (3.4), if you want to change how the screens are composed.
- A first pass at the brand question in section 9 — what happens to the wordmark in each direction.

Johannes picks one, or asks for a hybrid. **Then** you go deep.

This stage exists because a full redesign that goes the wrong way is expensive to discover at screen twenty. Two or three loose directions cost a fraction of one resolved screen, and they are the cheapest decision-making tool available.

7.2 Stage 1 — the pilot

The chosen direction, resolved properly, on the same two screens plus everything global:

- **Committing money** (**08-choose-stake** today) — a decision screen with a hero number, a repeating data-driven list, a modal, and four states.
- **Coming back** (**16-home** today) — the densest surface in the app, mixing static and live content, with an empty state.
- The **logo set**, the **icon set** and both **token sheets** — because they touch every screen, and discovering the icon export spec is wrong on screen 20 is a disaster while discovering it on screen 2 is a Tuesday.

We build both end to end, then sit down and fix whatever this brief got wrong — which fields were unclear, what was missing, where the format broke down. The brief is amended. **Then** the remaining screens are drawn.

The two gates together cost two or three weeks. Skipping them risks a wrong direction, or a format problem, replicated across the whole app.

7.3 Waves

Phase	You	Engineering	Gate
0	2-3 directions on 2 screens, rationale, flow proposal, brand first pass	Token architecture, lint ratchet, icon shell, font swap	A direction is chosen
1	Pilot: the chosen direction resolved, plus logo, icons, token sheets	Build the two pilot screens	Both accepted under section 6
2	Format retro	Brief amended, palette applied	Agreed changes to this document
3	Wave A onboarding · Wave B play loop · Wave C main app · Wave D chrome and legal	Build wave by wave	Section 6 per wave
4	Review fixes	Desktop backdrop, accessibility sweep, asset weight budget	Full acceptance

Delivering in waves is what lets building start on onboarding while the play loop is still being drawn. A single drop at the end wastes half the calendar.

8. What is happening in the codebase while you design

One thing, and it is worth knowing because it affects how cheaply your work gets applied.

The engineering side is putting the app onto the website's token architecture: adopting `getlucky-www`'s naming (`--green` , `--green-light` , `--charcoal`), collapsing the app's two duplicate token declarations into one `:root` block the Tailwind theme derives from, removing dead references — including `--font-heading` , which the website defines and the app only ever pointed at — and moving the display font to the `.woff2` the website already uses, 28 KB against the 97 KB `.otf` the app ships. An icon component goes in with today's emoji inside it, so your SVGs become a data swap rather than a refactor.

One deliberate change of plan. An earlier draft of this brief committed engineering to sweeping all ~350 hardcoded colour values out of the app before your comps arrived. That was right for a reskin. For a full redesign it is partly wasted work: a screen being rebuilt has its markup and styles rewritten anyway, so de-hardcoding it first is throwaway.

So the sweep is no longer front-loaded. Instead a lint rule fails the build on any new hardcoded colour, and each screen gets cleaned as it is rebuilt. The token layer is correct from day one; the old literals die as their screens do. Same destination, none of the wasted motion.

It runs while you draw, so it costs no calendar time. It also means the token sheet you fill in will actually govern the app, which today it would not.

9. The brand is open

Earlier drafts of this brief asked whether the palette was locked. It is not, and neither is the rest of it. **You may build on the current brand to make it work digitally.**

That phrase is doing precise work, so here is what it means in practice.

What is open

- **The palette.** The forest/gold/cream scheme in `design/01-tokens/tokens.csv` comes from the live website and is a considered, coherent starting point. Nothing in it is locked. Propose a different direction in the token sheet if V2 wants one.
- **The typography,** within the licensing note in `01-tokens/README.md`.
- **The wordmark itself.** The hand-lettered script is good, and it was drawn for print and signage — where it is never asked to survive at 24px, sit in a tab bar, animate, or share a row with an interface. You may redraw, simplify or restructure it so it works in a product.
- **What the brand *is* in a digital context** — how it behaves in motion, at small sizes, under a status bar, in a dark surface, on a loading state. Almost none of that has ever been decided.

What stays

It should still be Get Lucky. This is an evolution built on the existing brand, not a replacement of it — someone who knows the current mark should recognise the new one. We are not commissioning a rebrand, and a proposal that discards the equity in the script is the one thing here that would be out of scope.

The consequence, stated plainly

Whatever V2 lands on **becomes the brand**, and everything else trails it: the website at `getluckygolf.co.za`, signage, sponsor collateral, the bag tags. `docs/brand-guide.md` still carries an old rule that the live site wins where the site and the app disagree — that rule is retired.

So the app is now the place the brand gets decided. Design accordingly.

10. Logistics and next steps

Files. The `design/` folder is the structure. Work in whatever tool suits you and export into it. If you would rather work in a shared drive, mirror this exact folder structure and we will bring it into the repository — but the structure and the naming need to survive the trip, because they are what makes the handback buildable.

The website. `getluckyjo/getlucky-www` is the V2 base — its stylesheet, its logo files and the licensed display font are all copied into the design pack, and the live site is at `www.getluckygolf.co.za`. Look at it before you look at the app.

Access. You will be given a beta login. Use it — several screens read very differently with a real account than they do in the screenshots, and the five screens after the payment cannot be captured at all. Ask for a walkthrough of the play flow before designing those.

Questions. Anything ambiguous in this brief is a defect in the brief. Ask, and it gets amended rather than guessed at.

One thing not to do. Do not treat the current app as a design to improve incrementally. It is a working prototype whose look was never authored. Start from the product and the brand, not from those screenshots.

Next steps

1. Kick-off call.
2. Read `design/README.md` , then `design/02-screens/_EXAMPLE.md` .
3. Look through `design/00-reference/current-app/` — as a functional reference, not a design.
4. **Stage 0:** two or three visual directions on two screens, a rationale for each, the flow proposal if you want one, and a first pass at the wordmark question.
5. Direction chosen. **Stage 1:** the pilot — those two screens resolved, plus the logo set, the icon set and both token sheets.
6. Format retro, brief amended, remaining screens released in waves.